

Pet Health and Welfare Database: Final Project Report

Team Databytes

By: Justin Tran, Brandon Trinh, Andy Do



Table of Contents

Project Overview.....	4
Project Introduction.....	4
Changes to Original Proposal.....	4
Key Design Changes.....	4
Team Responsibilities.....	4
Schedule.....	5
Database Design.....	6
Entity-Relationship Diagram.....	6
ER-Level Design Decisions.....	6
Relational Model.....	7
RM-Level Constraints.....	8
RM-Level Design Decisions.....	12
Assumptions.....	12
Limitations.....	13
Implementation.....	14
Tooling Assessment.....	14
Railway Hosting Setup.....	15
Database Creations.....	15
Data Population Strategy.....	16
Tables Included in the Database.....	17
Data Validation & Testing.....	17
Functional Testing (Queries, API, and UI).....	17
Bad Data for Tests.....	18
Database Testing Screenshots.....	18
SQL Queries.....	21
SQL Query Statements.....	21
Use Case Queries.....	23
Normalization.....	23
User Interface & Demo.....	25
User Interface & Demo Scenarios.....	25
UI Functionality & CRUD Demonstration.....	26

Project Evaluation & Reflection.....	31
References.....	32
Appendices.....	33
Appendix A — Full database schema.....	33
Appendix B — All INSERT statements.....	35
Appendix C — All bad data tests.....	41
Appendix D — All SQL queries used in UI + demo.....	42

Project Overview

Project Introduction

Our team will develop a database that tracks canine and feline welfare outcomes across animal shelters in the United States. The database will store information about each organization, the animals they care for, and important events such as intakes, outcomes, and adoptions. It will help show how animals enter and leave shelters, for example, through owner surrenders, stray intakes, adoptions, transfers, or euthanasia. By organizing this data, we can identify trends between species, compare adoption rates by state, and analyze live release outcomes. The goal of this project is to help shelters and policymakers better understand animal outcomes and make informed decisions that improve the welfare of cats and dogs in shelters. We implemented a minimal web interface using React, FastAPI, and a Railway-hosted MySQL database to demonstrate end-to-end functionality.

Changes to Original Proposal

Our original proposal aimed to track outcomes for all dog and cat shelters across the United States. During development, we recognized that this scope was too large for the timeframe. In Iteration 3, we focused on a representative subset of organizations and animals. This allowed us to fully implement our schema, populate it with realistic sample data, and test constraints.

Key Design Changes

- Added Location as a core entity linked to Intake, Adopter, and Organization to support geographic tracking.
- Added WelfareExam to record animal health data over time.
- Redesigned Intake by removing the surrogate key and using a composite key (IntakeNo, AnimalID) to correctly model multiple intake events.
- Updated Outcome to include Returned and IsLive directly, removing the OutcomeType table.
- Modeled Adoption as a specialization of Outcome, using OutcomeID as both primary key and foreign key.

Team Responsibilities

Our whole team will:

- Attend and participate in all scheduled meetings
- Contribute to the project with equal amounts of work
- Communicate our progress, issues, or delays in the group chat
- Review and provide constructive feedback on all iterations before submission
- Collaborate on database testing and refinement
- Participate in group reviews and prepare materials for the final presentation
- Support teammates in debugging and improving project efficiency

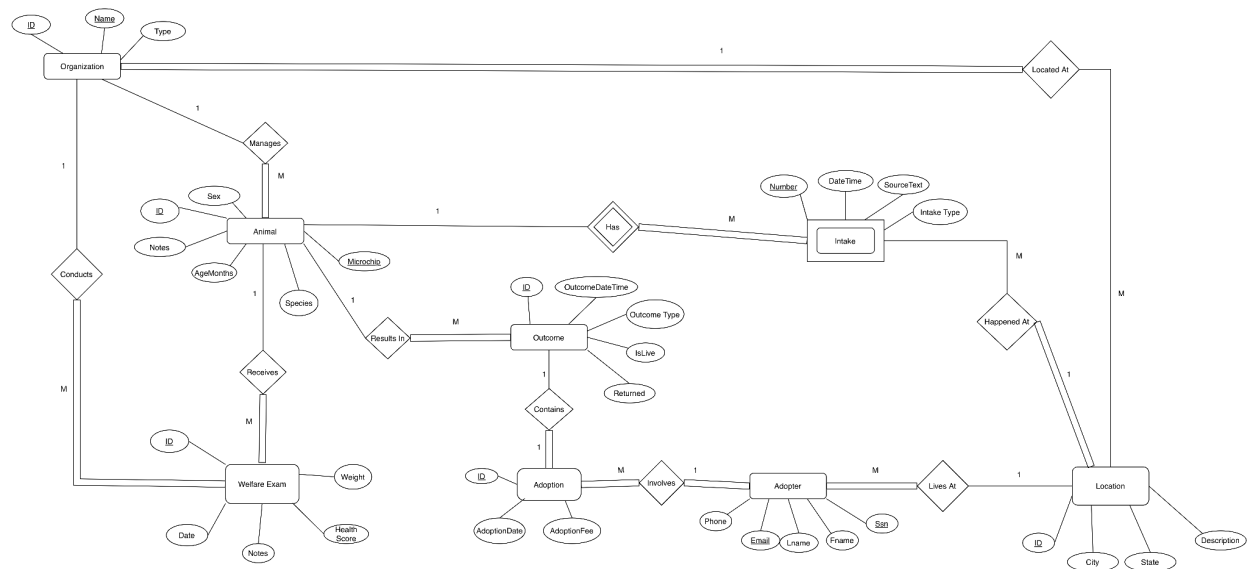
Schedule

Week	Milestone	Deliverables	Due Date
Week 1(Oct 27- Nov 2)	Scope Definition and Tool Setup	1. Finalize our scope 2. Choose DBMS, IDE, and hosting platform	1. Oct 30 2. Oct 31 Final: Nov 2
Week 2(Nov 3 - Nov 9)	ER and Relational Model	1. Build the ER diagram 2. Build the relational model	1. Nov 5 2. Nov 7 Final: Nov 9
Week 3(Nov 10 - Nov 16)	Database Creation and Display Queries	1. Create a database with the sample data 2. Create queries	1. Nov 12 2. Nov 14 Final: Nov 16
Week 4(Nov 17 - Nov 23)	Update Queries and Sample Data	1. Update our queries 2. Populate the database with our sample data	1. Nov 19 2. Nov 21 Final: Nov 23
Week 5(Nov 24 - Nov 30)	Testing and Refinement	1. Test all queries 2. Refine our documentation	1. Nov 26 2. Nov 28 Final: Nov 30

Week 6(Dec 1 - Dec 9)	Final Report and Presentation	1. Create the final report 2. Practice presentation and demo 3. Submit the final report and present our key findings	1. Dec 4 2. Dec 6 3. Dec 9
-----------------------	-------------------------------	--	--

Database Design

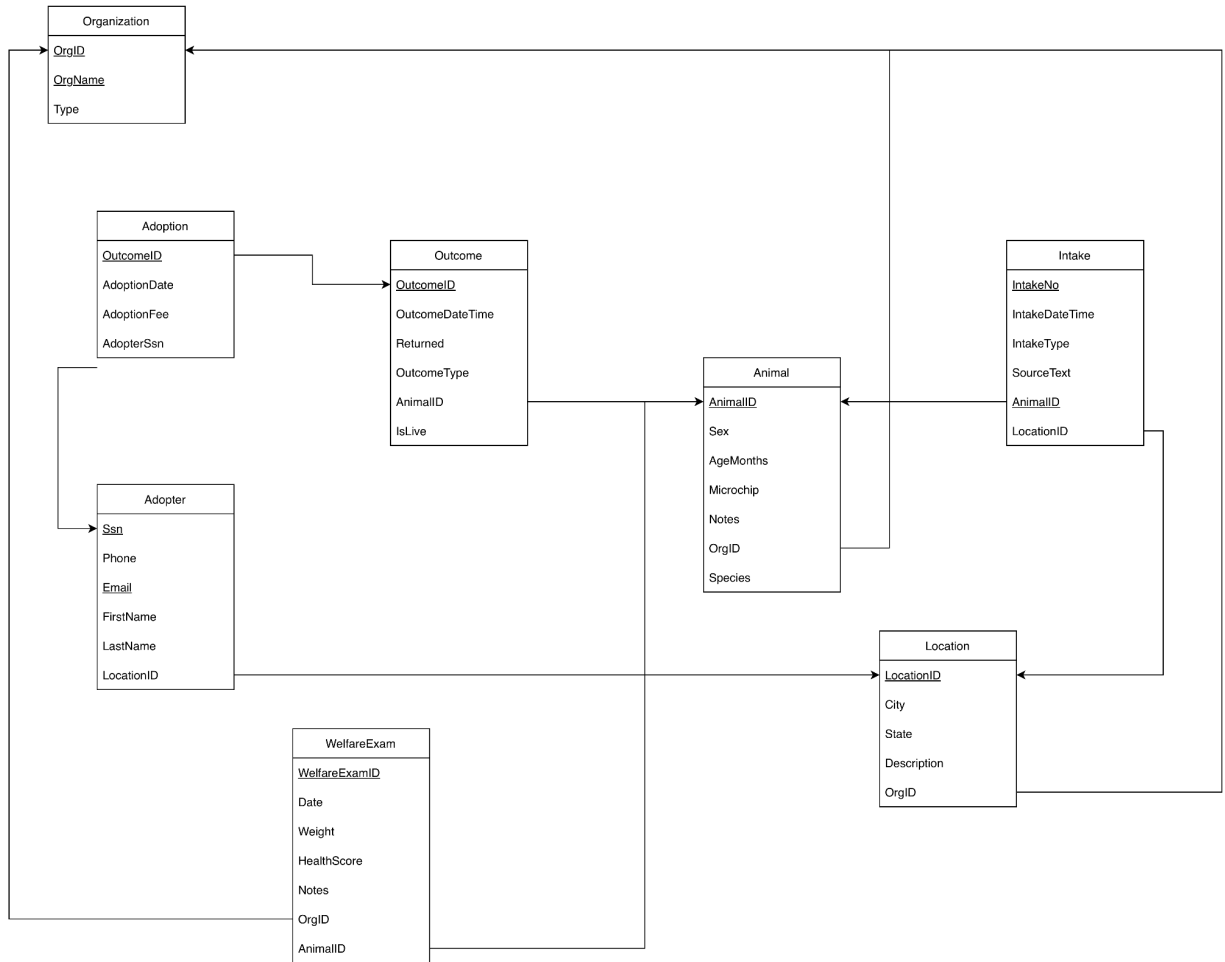
Entity-Relationship Diagram



ER-Level Design Decisions

- We modeled Intake as a weak entity under Animal using a composite primary key (IntakeNo, AnimalID). This allows a single animal to have multiple intake events over time, which is necessary to correctly represent real shelter behavior such as returns, transfers, and repeat stays.
- We store OutcomeType as a descriptive attribute instead of a lookup table and include IsLive directly in the Outcome table. This simplifies querying for survival outcomes and reduces unnecessary joins while still supporting clear classification of outcome events.
- Adoption is modeled as a specialization of Outcome. Every adoption must correspond to exactly one outcome, enforced by using OutcomeID as both the primary key and foreign key in Adoption. This guarantees that adoption records cannot exist without a corresponding outcome.
- We separated the Adopter into its own entity to avoid duplicating contact data across multiple adoptions. This ensures data consistency for repeat adopters.
- We introduced a Location entity that is shared by Intakes and Adopters. This enables geographic analysis of where animals are found and where adoptions occur.
- WelfareExam was added to track health evaluations over time. This allows long-term medical trend analysis per animal and per organization.
- Microchip is enforced as UNIQUE when present, ensuring no two animals share the same microchip ID while still allowing NULL values for animals without chips.

Relational Model



RM-Level Constraints

D_ID	INTEGER, >=1
D_SSN	CHAR(9), digits only
D_NAME	VARCHAR(100), printable characters
D_TEXT	VARCHAR(500), printable characters
D_EMAIL	VARCHAR(150), must contain @ and domain
D_PHONE	VARCHAR(20), digits/space/+/-/()
D_CITY	VARCHAR(50)
D_STATE	CHAR(2), USPS code
D_ZIP	VARCHAR(10) NNNNN or NNNNN-NNNN

D_SPECIES	VARCHAR(30)
D_SEX	CHAR(1), {M, F, U}
D_AGE_MONTHS	INTEGER, >= 0 <= 1000
D_MICROCHIP	VARCHAR(20), alphanumeric (unique)
D_BOOL	BOOLEAN (TRUE/FALSE)
D_MONEY	NUMERIC(8,2) 0.00-999999.99
D_TS	TIMESTAMP, YYYY-MM-DD HH:MM[:SS]
D_DATE	DATE, YYYY-MM-DD
D_WEIGHT	DECIMAL(5,2), >= 0.00
D_HEALTH_SCORE	INTEGER, between 1 and 10 (or NULL if not recorded)

Organization

OrgID	OrgID $\subset$ D_ID (PK, NOT NULL)
OrgName	OrgName $\subset$ D_NAME (UNIQUE, NOT NULL)
Type	Type $\subset$ D_NAME (NOT NULL)

Animal

AnimalID	AnimalID $\subset$ D_ID (PK, NOT NULL)
OrgID	OrgID $\subset$ D_ID (FK -> Organization.OrgID, NOT NULL)
Species	Species $\subset$ D_SPECIES (NOT NULL)
Sex	Sex $\subset$ D_SEX (NOT NULL)
AgeMonths	AgeMonths $\subset$ D_AGE_MONTHS (NOT NULL)
Microchip	Microchip $\subset$ D_MICROCHIP (NULL allowed)
Notes	Notes $\subset$ D_TEXT (NULL)

Intake (weak entity under animal)

IntakeNo	IntakeNo $\subset$ D_ID (part of composite PK, NOT NULL)
AnimalID	AnimalID $\subset$ D_ID (part of composite PK, FK $\rightarrow$ Animal.AnimalID, NOT NULL)
IntakeType	IntakeType $\subset$ D_NAME (NOT NULL) (e.g., "Stray", "Owner Surrender", "Public Assist")
IntakeDateTime	IntakeDateTime $\subset$ D_TS (NOT NULL)
SourceText	SourceText $\subset$ D_TEXT (NULL)
LocationID	LocationID $\subset$ D_ID (FK $\rightarrow$ Location.LocationID, NOT NULL)

Outcome

OutcomeID	OutcomeID $\subset$ D_ID (PK, NOT NULL)
AnimalID	AnimalID $\subset$ D_ID (FK $\rightarrow$ Animal.AnimalID, NOT NULL)
OutcomeType	OutcomeType $\subset$ D_NAME (NOT NULL)
OutcomeDateTime	OutcomeDateTime $\subset$ D_TS (NOT NULL)
Returned	Returned $\subset$ D_BOOL (NOT NULL DEFAULT FALSE)
IsLive	IsLive $\subset$ D_BOOL (NOT NULL) (TRUE if the animal left shelter alive)

Adopter

Ssn	Ssn $\subset$ D_SSN (PK, NOT NULL)
Phone	Phone $\subset$ D_PHONE (NOT NULL)
Email	Email $\subset$ D_EMAIL (NOT NULL)
FirstName	First $\subset$ D_NAME (NOT NULL)
LastName	Last $\subset$ D_NAME (NOT NULL)

LocationID	LocationID $\subset$ D_ID (FK $\rightarrow$ Location.LocationID, NOT NULL)
------------	--

Adoption (specialization of Outcome)

OutcomeID	OutcomeID $\subset$ D_ID (PK, FK $\rightarrow$ Outcome.OutcomeID, NOT NULL)
AdopterSsn	AdopterSsn $\subset$ D_SSN (FK $\rightarrow$ Adopter.Ssn, NOT NULL)
AdoptionDate	AdoptionDate $\subset$ D_DATE (NOT NULL)
AdoptionFee	AdoptionFee $\subset$ D_MONEY (NOT NULL DEFAULT 0.00)

Location

LocationID	LocationID $\subset$ D_ID (PK, NOT NULL)
City	City $\subset$ D_CITY (NOT NULL)
State	State $\subset$ D_STATE (NOT NULL)
Description	Description $\subset$ D_TEXT (NULL ALLOWED)
OrgID	OrgID $\subset$ D_ID (FK $\rightarrow$ Organization.OrgID, NOT NULL)

WelfareExam

WelfareExamID	WelfareExamID $\subset$ D_ID (PK, NOT NULL)
Date	Date $\subset$ D_DATE (NOT NULL)
Weight	Weight $\subset$ D_WEIGHT (NULL ALLOWED)
HealthScore	HealthScore $\subset$ D_HEALTH_SCORE (NULL ALLOWED)
Notes	Notes $\subset$ D_TEXT (NULL ALLOWED)
OrgID	OrgID $\subset$ D_ID (FK $\rightarrow$ Organization.OrgID, NOT NULL)
AnimalID	AnimalID $\subset$ D_ID (FK $\rightarrow$ Animal.AnimalID, NOT NULL)

RM-Level Design Decisions

Primary Keys and Entity Modeling

- Most tables use a single-column surrogate integer primary key (OrgID, LocationID, AnimalID, OutcomeID, WelfareExamID) to simplify joins and maintain consistency in the schema.
- Intake is modeled as a weak entity with the composite primary key (IntakeNo, AnimalID). This supports multiple intake events per animal without duplicating animal data.
- Adopter uses SSN as a natural primary key, reflecting real-world shelter workflows where adopter identity is tracked using government-issued ID.
- Adoption uses OutcomeID as both its primary key and a foreign key to Outcome. This models adoption as a specialization of an outcome event and ensures that an outcome can be adopted at most once.

Foreign Keys and Relationships

- Animal includes OrgID as a foreign key to ensure that each animal belongs to exactly one organization.
- Intake includes foreign keys to AnimalID and LocationID to ensure each intake event belongs to a specific animal and occurs at a specific shelter location.
- IntakeType is stored as an attribute describing how the animal entered the system.
- Outcome includes AnimalID as a foreign key to link each outcome event to a specific animal and support multiple outcomes over time.
- Adoption enforces a relationship between OutcomeID and AdopterSsn to ensure that each adoption event is connected to exactly one outcome and one adopter.
- WelfareExam includes foreign keys to OrgID and AnimalID to tie each exam to the organization responsible for conducting it and the corresponding animal.

Attribute-Level Constraints and Data Validation

- Microchip is unique when it is not null. This prevents two animals from being assigned the same chip while still allowing records for animals that do not have one.
- AgeMonths is required to be zero or greater to avoid invalid negative age values.
- HealthScore is limited to values from 1 to 10 (or null), ensuring consistent scoring for welfare exams.
- Outcome includes two boolean fields, IsLive and Returned, which make it easier to filter for live-release outcomes and to track whether adopted animals were later returned.
- We also follow a timeline rule in which outcomes should happen after the corresponding intake event. MySQL cannot enforce this as a cross-table constraint, but we maintain this rule through data validation and testing.

Assumptions

- The database tracks only dogs and cats from U.S. shelters, using a two-letter state code.
- Each animal is managed by exactly one organization at a time.
- All intakes, outcomes, and adoptions must reference an existing animal record.

- IntakeType and OutcomeType are stored as descriptive text values rather than in separate lookup tables.
- Each adoption corresponds to exactly one outcome and one adopter.
- Adoption records appear only for outcomes whose type represents an adoption event.
- Microchip numbers are unique when present but may be null for animals without microchips.
- OutcomeDateTime is expected to occur on or after the relevant intake event; this rule is maintained through validation rather than enforced through SQL constraints.
- The Returned field on Outcome indicates whether an adopted animal was brought back to the shelter.
- Adopters are stored once using SSN as their primary identifier.
- All locations associated with shelter operations are stored in the Location table and belong to one organization.
- AgeMonths must be non-negative and within a realistic range.
- Welfare exams may occur multiple times for the same animal and are tied to a single organization and animal.
- All timestamps are treated as if they are in the same time zone.

Limitations

- The database focuses only on dogs and cats, excluding other animal species such as birds, reptiles, or livestock.
- Timestamp data does not account for different time zones across organizations, which may slightly affect time-based comparisons between shelters in different regions.
- Shelter records may vary in naming conventions, causing potential inconsistencies in IntakeType and OutcomeType text values since these are stored as descriptive attributes rather than enforced lookup tables.
- Financial details such as adoption fees are simplified and may not reflect real-world variability such as sliding scale pricing, promotions, or fee waivers.
- The model assumes valid and unique IDs, SSNs, and microchip numbers, which may not always be guaranteed in real-world data collection environments.
- Breed, color, and behavioral training history are not modeled, limiting more detailed animal profiling.
- Medical records are simplified to welfare exams only and do not capture full treatment plans, prescriptions, or diagnostic imaging.
- The model does not track inter-organization transfers as a separate entity, only as outcome records.
- User authentication, staff accounts, and role-based access control are not implemented, as the focus of the project is database design rather than application security.
- The system does not enforce automatic validation that OutcomeDateTime always follows the most recent IntakeDateTime at the SQL level, relying instead on application logic.

Implementation

This section describes the tools used to build, deploy, and test the database.

Tooling Assessment

DBMS: MySQL

We chose MySQL because it is beginner-friendly, widely supported, and easier to work with in teams that are both new to database design and SQL. Our group has limited prior experience with relational databases, and MySQL offers a smoother learning experience compared to more advanced systems like PostgreSQL. It supports foreign keys, constraints, indexing, and structured queries while also remaining easy to integrate with Python.

Backend: FastAPI (Python)

Handles data requests, runs SQL queries, & talks to the database

FastAPI will serve as our backend, handle API requests, and send SQL queries to our database. It is beginner-friendly because it is easy to route and run queries, and it automatically generates a /docs page where we can test our API without needing JavaScript. FastAPI also integrates seamlessly with Python, which our group is comfortable with. This makes it easier to focus more on how to connect the database to a web interface.

Frontend: React + Vite (Deployed on Vercel)

Our application was built with React. We chose React because it is easy to fetch and display live data from our FastAPI backend. React also supports reusable UI components. It allows for dynamic filtering, forms, and interactive CRUD functionality. And it is easy to deploy with Vercel. With Vercel, it automatically builds and hosts our UI and gives us a public URL for presentation and demo.

The frontend demonstrates:

- Reading animals from the database
- Filtering (client-side)
- Adding new animals
- Deleting animals
- Running complex queries with dropdown filters

IDE: VS Code

Our team is comfortable with VS Code, and it supports:

- Python & FastAPI development
- React development
- MySQL extensions

- Git/GitHub integration
- Virtual environment support

Hosting: Railway (Cloud MySQL Instance)

We originally planned to host our MySQL database on Microsoft Azure, but we ran into connection problems that prevented us from using it. Azure repeatedly timed out when we attempted to connect through MySQL Workbench or from our backend API, even after adjusting firewall settings and testing multiple connection strings. Because we could not establish an external connection to the Azure instance, we were unable to load data or run queries during development.

Because of these issues, we switched to Railway, which provides a MySQL connection string and does not require custom network or firewall configuration. Railway allowed MySQL Workbench and our FastAPI backend to connect on the first attempt. We now host our database on Railway and use Workbench to test queries and verify constraints. Our FastAPI backend is also deployed on Railway, allowing both the frontend and backend to run reliably in the cloud.

Other Tools

- Draw.io: ERD Visualization
- Excel: cleaning imported shelter data and preparing CSVs
- GitHub: version control and collaborative work across iterations
- MySQL Workbench: testing queries, debugging constraints, populating sample data

Railway Hosting Setup

To host our database in the cloud, we deployed a MySQL instance using Railway's built-in MySQL service. Railway created the railway database and provided the connection string (host, port, username, password). In MySQL Workbench, we connected to this instance by creating a new connection using the given credentials. After connecting, we ran our CREATE TABLE and INSERT statements to deploy the schema with sample data. To ensure that deployment worked, we ran SHOW TABLE; and several SELECT queries. The screenshots below show that the schema is hosted on the cloud, sample data, and constraint enforcements.

Database Creations

Example 1: Organization Table

```
-- 1. ORGANIZATION
CREATE TABLE Organization (
  OrgID      INT          NOT NULL,
  OrgName    VARCHAR(100) NOT NULL,
  Type       VARCHAR(100) NOT NULL,
  CONSTRAINT PK_Organization PRIMARY KEY (OrgID),
  CONSTRAINT UQ_Organization_OrgName UNIQUE (OrgName),
  CONSTRAINT CK_Organization_OrgID CHECK (OrgID >= 1)
);
```

Example 2: Animal Table

```
-- 3. ANIMAL
CREATE TABLE Animal (
  AnimalID INT NOT NULL,
  OrgID INT NOT NULL,
  Species VARCHAR(30) NOT NULL,
  Sex CHAR(1) NOT NULL,
  AgeMonths INT NOT NULL,
  Microchip VARCHAR(20) NULL,
  Notes VARCHAR(500) NULL,
  CONSTRAINT PK_Animal PRIMARY KEY (AnimalID),
  CONSTRAINT FK_Animal_Org
    FOREIGN KEY (OrgID) REFERENCES Organization (OrgID),
  CONSTRAINT UQ_Animal_Microchip UNIQUE (Microchip),
  CONSTRAINT CK_Animal_AnimalID CHECK (AnimalID >= 1),
  CONSTRAINT CK_Animal_Sex CHECK (Sex IN ('M', 'F', 'U')),
  CONSTRAINT CK_Animal_AgeMonths CHECK (AgeMonths BETWEEN 0 AND 1000)
);
```

The Organization and Animal tables demonstrate our use of surrogate primary keys (OrgID, AnimalID) to maintain consistency and simplify joins across the database. The Organization table uses a UNIQUE constraint on OrgName, while the Animal table includes additional constraints on species, sex, age, and microchip IDs, ensuring data accuracy and preventing invalid entries. A foreign key from Animal to Organization maintains referential integrity and guarantees that each animal belongs to a valid shelter. Both tables satisfy BCNF, since all attributes depend only on their primary key and no transitive dependencies exist. All remaining CREATE TABLE statements are included in Appendix A.

Data Population Strategy

To populate the database, we created a structured set of sample data designed to support the project's user scenarios and demonstrate meaningful query results. For each major entity (Animal, Intake, Outcome, Adoption, Adopter, Organization, Location, and WelfareExam), we added realistic records that reflect common animal shelter operations. The data includes variation in species, organizations, locations, and outcome types so that multi-table joins and aggregate queries return informative results.

The dataset was intentionally kept small but comprehensive. Rather than focusing on volume, we focused on ensuring that the data exercised key relationships, constraints, and use cases. Some records were designed to represent special cases, such as animals with multiple outcomes, bonded pairs, and return scenarios, to validate one-to-many relationships and schema behavior.

Sample data was generated manually, with the assistance of ChatGPT to create realistic animal descriptions and scenarios. All generated data was reviewed and adjusted to ensure that foreign keys were valid, constraints were respected, and queries produced expected results. No real or external shelter data was used.

To keep the main report concise, only two representative INSERT statements are shown below. The complete set of SQL statements used to populate the database is provided in Appendix B.

Example INSERT Animal:

```
INSERT INTO Animal (AnimalID, OrgID, Species, Sex, AgeMonths, Microchip, Notes)
VALUES
(1000, 1, 'Dog', 'F', 18, 'MC1000A', 'Friendly, came in as stray.');
```

Example INSERT Outcome:

```
INSERT INTO Outcome (OutcomeID, AnimalID, OutcomeType, OutcomeDateTime, Returned,
IsLive)
VALUES
(2000, 1000, 'Adopted', '2025-01-20 14:00:00', FALSE, TRUE);
```

Tables Included in the Database

The following tables are used in our project:

- Organization
- Location
- Animal
- Intake
- Outcome
- Adoption
- Adopter
- WelfareExam

Data Validation & Testing

Functional Testing (Queries, API, and UI)

We tested the system across all layers (the MySQL database, the FastAPI backend, and the React UI) to make sure everything worked correctly from end to end.

API and Endpoint Testing:

Each FastAPI endpoint was tested using the automatically generated /docs page and direct requests. We verified that responses from endpoints such as /animals-simple matched the results of the corresponding SQL queries when run directly in MySQL Workbench. CRUD

Operation Testing:

Create, update, and delete operations were first tested directly in MySQL to confirm correct behavior and constraint enforcement. We then tested the same operations through the UI and

verified that UI actions correctly modified the database and returned updated results through the API.

Complex Query Testing:

All complex queries included in the report were tested using the populated sample data. We manually checked results across related tables (such as Animal, Outcome, Adoption, Location, and Adopter) to ensure joins, counts, and aggregations produced correct and expected results.

UI and Backend Integration Testing: We tested the live deployed UI on Vercel to confirm proper communication with the FastAPI backend hosted on Railway. Data loading, filtering, and updates were verified to ensure that the UI remained consistent with the database.

Overall, these tests confirmed that the database, backend, and UI worked together correctly and that the system behaved as expected.

Bad Data for Tests

To verify that our schema behaves correctly, we intentionally executed SQL statements that violate:

- **Unique constraints** (duplicate microchip values).
- **Domain constraints** (invalid sex values).
- **Foreign key constraints** (references to non-existent animals or outcomes).
- **Referential integrity** (attempting to delete an outcome that is used by an adoption).

Each of these constraints produced an error in MySQL, confirming that our database creation and constraints work together to reject invalid or inconsistent data.

Database Testing Screenshots

We validated our design by running both valid and invalid SQL statements against the Railway-hosted MySQL instance. Valid inserts confirmed that foreign key relationships and domain constraints were satisfied, while invalid test cases produced errors as expected. The figures below show our connection, sample data, and example constraint enforcement.

Figure 1: Connection to Railway-hosted database in MySQL Workbench.

SHOW TABLES;

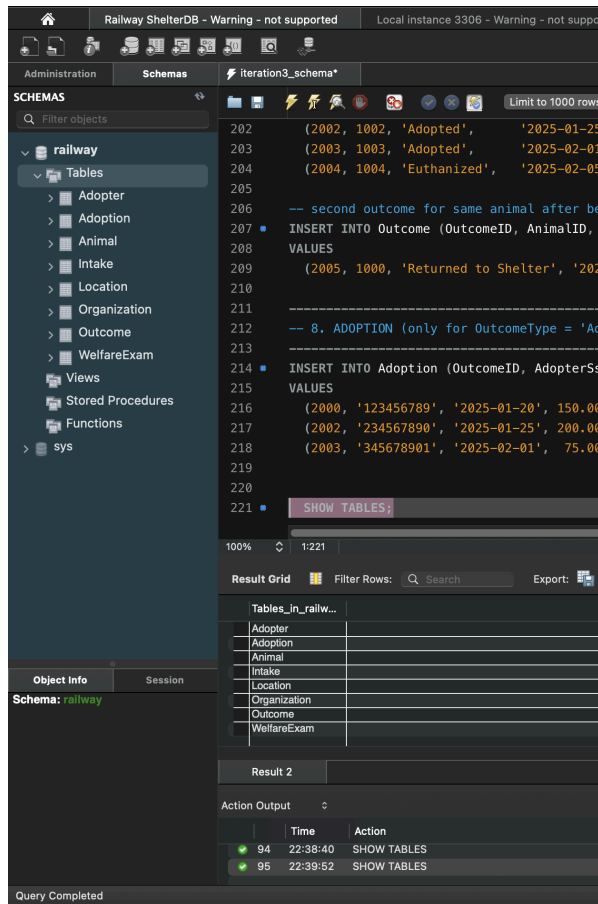


Figure 2: Sample data in the animal table.
 SELECT * FROM Animal;

221 **SELECT * FROM Animal;**

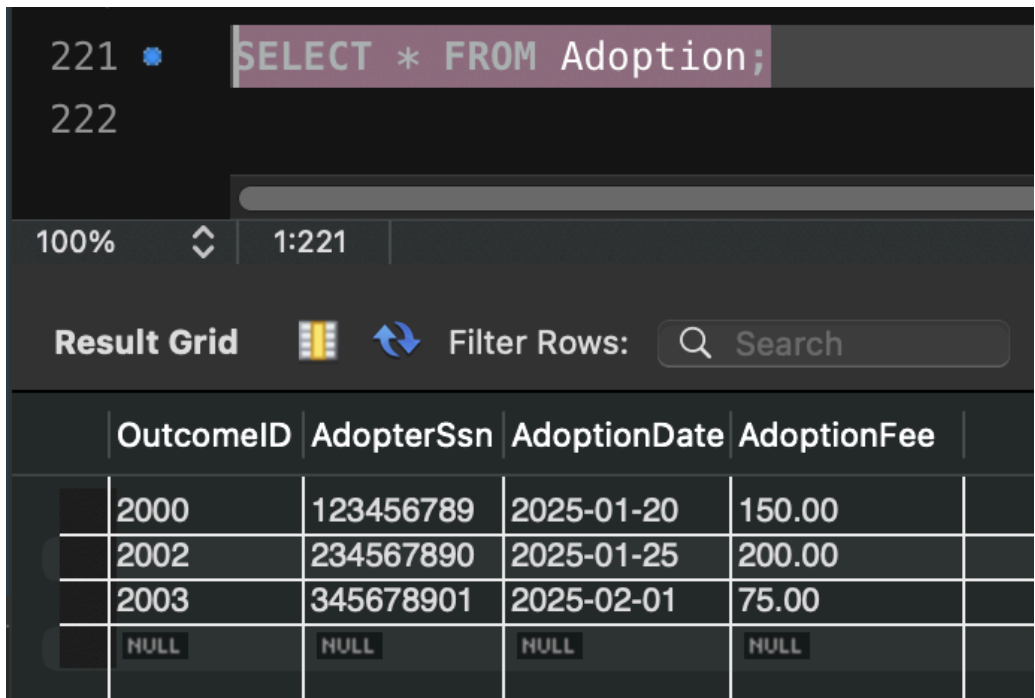
222

100% 22:221

Result Grid Filter Rows: Search Edit: Export/Import

	AnimalID	OrgID	Species	Sex	AgeMonths	Microchip	Notes
	1000	1	Dog	F	18	MC1000A	Friendly, came in as stray.
	1001	1	Cat	M	60	MC1001B	Owner surrender; shy in kennel.
	1002	1	Dog	M	6	NULL	Found in park with no collar.
	1003	2	Cat	F	36	MC1003D	Transferred from another rescue.
	1004	2	Dog	U	120	MC1004E	Senior dog with mobility issues.
	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figure 3: Adoption records demonstrating specialization of outcome.
SELECT * FROM Adoption;



221 SELECT * FROM Adoption;

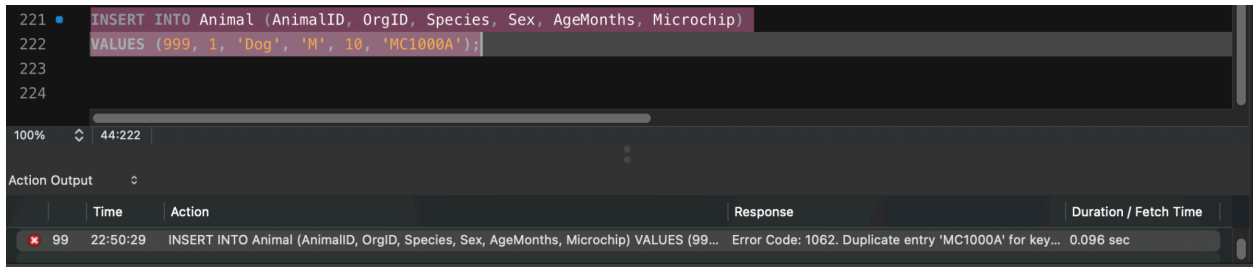
222

100% 1:221

Result Grid Filter Rows: Search

	OutcomeID	AdopterSsn	AdoptionDate	AdoptionFee
	2000	123456789	2025-01-20	150.00
	2002	234567890	2025-01-25	200.00
	2003	345678901	2025-02-01	75.00
	NULL	NULL	NULL	NULL

Figure 4: Failed insert demonstrating constraint enforcement.
INSERT INTO Animal (AnimalID, OrgID, Species, Sex, AgeMonths, Microchip)
VALUES (999, 1, 'Dog', 'M', 10, 'MC1000A');



221 INSERT INTO Animal (AnimalID, OrgID, Species, Sex, AgeMonths, Microchip)

222 VALUES (999, 1, 'Dog', 'M', 10, 'MC1000A');

223

224

100% 44:222

Action Output

	Time	Action	Response	Duration / Fetch Time
99	22:50:29	INSERT INTO Animal (AnimalID, OrgID, Species, Sex, AgeMonths, Microchip) VALUES (99...	Error Code: 1062. Duplicate entry 'MC1000A' for key...	0.096 sec

(Error Code: 1062. Duplicate entry 'MC1000A' for key 'Animal.UQ_Animal_Microchip')

Figure 5: JOIN query demonstrating relationship between Animal and Outcome.
SELECT
a.AnimalID,
a.Species,
o.OutcomeType,
o.OutcomeDateTime
FROM Animal a
JOIN Outcome o ON a.AnimalID = o.AnimalID
ORDER BY o.OutcomeDateTime DESC;

```

221 • SELECT
222     a.AnimalID,
223     a.Species,
224     o.OutcomeType,
225     o.OutcomeDateTime
226 FROM Animal a
227 JOIN Outcome o ON a.AnimalID = o.AnimalID
228 ORDER BY o.OutcomeDateTime DESC;
229

```

100% 33:228

Result Grid Filter Rows: Search Export:

	AnimalID	Species	OutcomeType	OutcomeDateTime
	1000	Dog	Returned to Shelter	2025-02-10 11:00:00
	1004	Dog	Euthanized	2025-02-05 17:20:00
	1003	Cat	Adopted	2025-02-01 11:00:00
	1002	Dog	Adopted	2025-01-25 16:45:00
	1000	Dog	Adopted	2025-01-20 14:00:00
	1001	Cat	Transferred	2025-01-18 09:30:00

SQL Queries

SQL Query Statements

The table below summarizes the main SQL statements used in our project and explains what each one does and which user scenarios it supports. The parameter ? is intended to be filled in by the UI or API layer at runtime. A complete list of all SQL queries (including variations and filters) is provided in Appendix D.

SQL Statement	Purpose
SELECT a.AnimalID, a.Species, a.Sex, a.AgeMonths, o.OutcomeType, o.OutcomeDateTime FROM Animal a JOIN Outcome o ON a.AnimalID = o.AnimalID	Shows animals with their outcomes, ordered by outcome date. Supports staff reviewing recent outcomes during the demo.

ORDER BY o.OutcomeDateTime DESC;	
SELECT I.State, a.Species, COUNT(*) AS AdoptionCount FROM Adoption ad JOIN Outcome o ON ad.OutcomeID = o.OutcomeID JOIN Animal a ON o.AnimalID = a.AnimalID JOIN Adopter d ON ad.AdopterSsn = d.Ssn JOIN Location l ON d.LocationID = l.LocationID GROUP BY I.State, a.Species;	Counts adoptions by state and species. Supports the “manager” or “policy” scenario where we compare adoption patterns geographically.
SELECT a.Species, SUM(CASE WHEN o.IsLive = TRUE THEN 1 ELSE 0 END) AS LiveOutcomes, COUNT(*) AS TotalOutcomes FROM Outcome o JOIN Animal a ON o.AnimalID = a.AnimalID GROUP BY a.Species;	Computes live outcomes vs total outcomes by species to estimate live-release performance. Supports analysis of outcome quality by species.
SELECT * FROM Animal WHERE Species = ? AND (OrgID = ? OR ? IS NULL);	Parameterized search that allows staff to filter animals by species and optionally by organization. Supports the main “animal search” interaction in the UI.
SELECT a.AnimalID, a.Species FROM Animal a WHERE a.AnimalID IN (SELECT o.AnimalID FROM Outcome o WHERE o.OutcomeType = 'Adopted');	Nested query that returns all animals that have at least one adoption outcome. Supports reporting on which animals have been successfully adopted.
INSERT INTO Animal (AnimalID, OrgID, Species, Sex, AgeMonths, Microchip, Notes) VALUES (?, ?, ?, ?, ?, ?, ?);	Inserts a new animal into the system as part of the intake process. Demonstrates CRUD “Create” functionality in the demo.
INSERT INTO Intake (IntakeNo, AnimalID, IntakeType, IntakeDateTime, SourceText, LocationID) VALUES (?, ?, ?, ?, ?, ?);	Records a new intake for an existing animal. Used when demonstrating how staff log new arrivals or returns to the shelter.
UPDATE Adopter SET Phone = ?, Email = ? WHERE Ssn = ?;	Updates adopter contact information when it changes. Demonstrates CRUD “Update” functionality.

DELETE FROM Adoption WHERE OutcomeID = ?;	Deletes a specific adoption record, for example if it was entered incorrectly. Demonstrates CRUD “Delete” functionality.
---	--

Use Case Queries

These use case questions are supported by the SQL statements summarized in the SQL Query Statements section and fully listed in Appendix D.

Select/Analysis:

- What is the number of adoption counts of each species by state?
- What are the total feline and canine intakes and outcomes per year by state?
- What are the total live outcomes by state and species?
- What is the average number of animals handled per organization by state and year?
- What is the year-by-year adoption and euthanasia rate by species of all states?

Insert/Delete:

- Insert a new animal and its first intake.
- Insert an adoption outcome.
- Update adopter contact information.
- Update/correct an intake type or timestamp.
- Delete an adoption record.

Normalization

This section evaluates each relation in our database to determine its normal form. All tables were analyzed for 1NF, 2NF, 3NF, and BCNF. Because the design uses surrogate primary keys and avoids multi-valued or derived attributes, the relations satisfy higher levels of normalization.

Organization(OrgID, OrgName, Type)

FD: {OrgID} → {OrgName, Type}

{OrgName} → {OrgID, Type} (OrgName is UNIQUE)

1NF: All attributes are atomic with no repeating groups.

2NF: The primary key is a single attribute (OrgID), so partial dependencies are not possible.

3NF: There are no transitive dependencies. All non-key attributes depend directly on the key.

BCNF: Every determinant (OrgID and OrgName) is a candidate key.

Conclusion: Organization is in BCNF.

Location(LocationID, City, State, Description, OrgID)

FD: {LocationID} → {City, State, Description, OrgID}

1NF: All attributes are atomic with no repeating groups.

2NF: The primary key is LocationID, so all attributes are fully dependent on the key.

3NF: No transitive dependencies exist within the table. OrgID is a foreign key but does not determine any other attributes in Location.

BCNF: All non-trivial functional dependencies have a superkey on the left side.

Conclusion: Location is in BCNF.

Animal(AnimalID, OrgID, Species, Sex, AgeMonths, Microchip, Notes)

FD: {AnimalID} → {OrgID, Species, Sex, AgeMonths, Microchip, Notes}

1NF: All attributes are atomic with no repeating groups.

2NF: The primary key is AnimalID, so no partial dependencies occur.

3NF: There are no transitive dependencies between non-key attributes.

BCNF: The primary key is the only determinant for all non-key attributes.

Conclusion: Animal is in BCNF.

Intake(IntakeNo, AnimalID, IntakeType, IntakeDateTime, SourceText, LocationID)

FD: {IntakeNo, AnimalID} → {IntakeType, IntakeDateTime, SourceText, LocationID}

1NF: All attributes are atomic with no repeating groups.

2NF: All non prime attributes depend on the full composite key, not just IntakeNo or AnimalID individually.

3NF: No transitive dependencies exist. LocationID does not determine other attributes in Intake.

BCNF: The composite primary key is the only determinant and is a superkey.

Conclusion: Intake is in BCNF.

Outcome(OutcomeID, AnimalID, OutcomeType, OutcomeDateTime, Returned, IsLive)

FD: {OutcomeID} → {AnimalID, OutcomeType, OutcomeDateTime, Returned, IsLive}

1NF: All attributes are atomic with no repeating groups.

2NF: OutcomeID is a single attribute primary key, so partial dependencies are impossible.

3NF: No transitive dependencies exist between non-key attributes.

BCNF: OutcomeID is the only determinant and is a candidate key.

Conclusion: Outcome is in BCNF.

Adoption(OutcomeID, AdopterSsn, AdoptionDate, AdoptionFee)

FD: {OutcomeID} → {AdopterSsn, AdoptionDate, AdoptionFee}

1NF: All attributes are atomic with no repeating groups.

2NF: The primary key is OutcomeID, so all attributes are fully dependent on it.

3NF: No transitive dependencies exist.

BCNF: OutcomeID is the only determinant and is a candidate key.

Conclusion: Adoption is in BCNF.

Adopter(Ssn, Phone, Email, FirstName, LastName, LocationID)

FD: {Ssn} → {Phone, Email, FirstName, LastName, LocationID}

1NF: All attributes are atomic with no repeating groups.

2NF: The primary key is Ssn, so partial dependencies do not occur.

3NF: There are no transitive dependencies among non-key attributes

BCNF: Ssn is the only determinant and is a candidate key.

Conclusion: Adopter is in BCNF.

WelfareExam(WelfareExamID, Date, Weight, HealthScore, Notes, OrgID, AnimalID)

FD: {WelfareExamID} → {Date,Weight,HealthScore,Notes,OrgID,AnimalID}

1NF: All attributes are atomic with no repeating groups.

2NF: The primary key is a single attribute, so no partial dependencies exist

3NF: No transitive dependencies exist within the table.

BCNF: WelfareExamID is the only determinant and is a candidate key.

Conclusion: WelfareExam is in BCNF.

Overall Summary

All tables in our schema satisfy 1NF, 2NF, 3NF, and, in practice, BCNF. This minimizes redundancy and avoids common update, insertion, and deletion anomalies while keeping the design simple to query and maintain.

Some Normalization Considerations

Although it meets BCNF, there could be some normalization done that would help it scale better. Columns like Species, OutcomeType, and IntakeType should be shifted to lookup tables. The large text columns, which include descriptions or notes, should be split into separate tables as well.

Although not necessary for this project, these improvements would be useful for enhancing data integrity, eliminating redundancy, and enabling more flexible system growth.

User Interface & Demo

User Interface & Demo Scenarios

For Iteration 4, we implemented a simple web-based UI to demonstrate that our Pet Health & Welfare database is accessible through a live backend. The frontend is built with React + Vite and deployed on Vercel, while our backend is implemented with FastAPI and hosted on Railway. The MySQL database is also hosted on Railway.

When the Animals page loads, the React app sends a GET request to /animals-simple. FastAPI connects to our Railway MySQL instance using SQLAlchemy and returns results as JSON. SQLAlchemy is only used to manage the database connection between FastAPI and the MySQL database; all queries executed in the project are written in raw SQL.

The UI displays this data in a table showing each animal's ID, species, sex, and age in months. We also include client-side species filtering (All / Dog / Cat) and a small summary of species

counts.

Although minimalistic, this UI demonstrates a complete end-to-end flow:
React UI → FastAPI backend → Railway MySQL database

It satisfies the requirement of showing CRUD operations and complex multi-table queries supported by our schema.

Our final documentation includes screenshots of the working UI and the SQL queries that support each feature. These demonstrate that our minimal interface can successfully perform CRUD operations and run the complex multi-table queries required for the project.

Deployed UI: <https://pet-welfare-ui.vercel.app/>

Backend API: <https://pet-welfare-backend-production.up.railway.app/docs>



Pet Health & Welfare – Animals View

This page pulls live data from our Railway MySQL database via the FastAPI backend.

Showing 31 animals Filter by species: All

Animals by species

Dog: 18 Cat: 13

ID	OrgID	Species	Sex	Age (months)	Actions
1000	1	Dog	F	18	Delete
1001	1	Cat	M	60	Delete
1002	1	Dog	M	6	Delete
1003	2	Cat	F	36	Delete
1004	2	Dog	U	120	Delete
1005	1	Dog	F	24	Delete
1006	1	Cat	F	8	Delete
1007	2	Dog	M	36	Delete
1008	3	Cat	M	48	Delete
1009	3	Cat	F	48	Delete
1010	4	Dog	F	4	Delete

UI Functionality & CRUD Demonstration

Our frontend is deployed on React + Vite and connects directly to the Railway-hosted MySQL database through a FastAPI backend. This provides real-time database interaction.

Read: View Animals Table

- Fetches live data from /animals-simple
- Allows filtering by species (Dog, Cat, ALL)
- Demonstrates SELECT queries and parametrized filtering

Pet Health & Welfare – Animals View

This page pulls live data from our Railway MySQL database via the FastAPI backend.

Showing 31 animals Filter by species: All

Animals by species

Dog: 18
Cat: 13

ID	OrgID	Species	Sex	Age (months)	Actions
1000	1	Dog	F	18	Delete
1001	1	Cat	M	60	Delete
1002	1	Dog	M	6	Delete
1003	2	Cat	F	36	Delete
1004	2	Dog	U	120	Delete
1005	1	Dog	F	24	Delete
1006	1	Cat	F	8	Delete
1007	2	Dog	M	36	Delete
1008	3	Cat	M	48	Delete

Create: Add a New Animal

The User can add a new animal using a form at the bottom of the page.

- **API Endpoint Called:** POST /add-animal
- **SQL Executed:** INSERT INTO Animal (AnimalID, OrgID, Species, Sex, AgeMonths, Microchip, Notes) VALUES (?, ?, ?, ?, ?, ?, ?)

Purpose: Creates a new animal record and immediately updates the UI, demonstrating CRUD “Create”.

Add New Animal (Create)

* AnimalID must be unique and OrgID must exist.

AnimalID

OrgID

Species

U

Age (months)

Add Animal

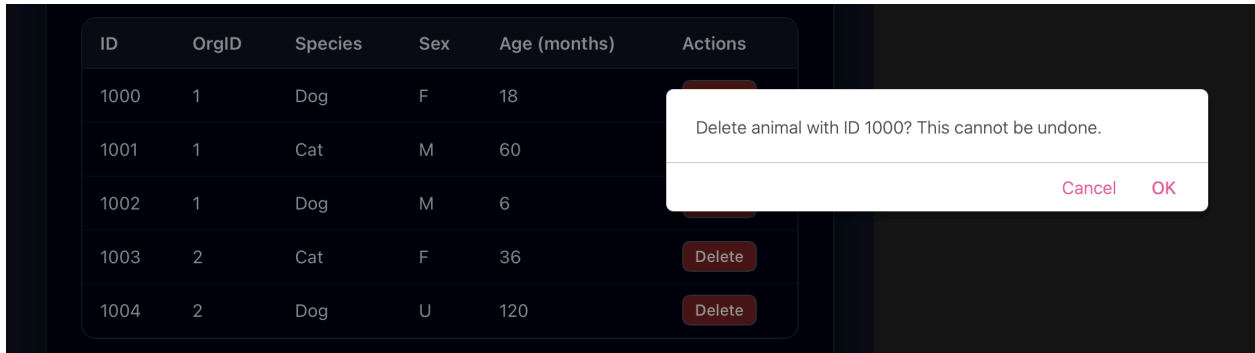
Data source: Pet Health & Welfare MySQL database on Railway.

Delete: Remove an Animal

Every row in the animal table includes a Delete button.

When Clicked:

- **API Endpoint Called:** DELETE /delete-animal{AnimalID}
- **SQL Executed:** DELETE FROM Animal WHERE AnimalID = ?;
- **Purpose:**
 - Allows staff to remove outdated or incorrect animal entries.
 - Demonstrating CRUD “Delete”.



The screenshot shows a table with columns: ID, OrgID, Species, Sex, Age (months), and Actions. The table contains five rows of animal data. A modal dialog is open, asking for confirmation to delete the animal with ID 1000. The dialog text is: "Delete animal with ID 1000? This cannot be undone." with "Cancel" and "OK" buttons.

ID	OrgID	Species	Sex	Age (months)	Actions
1000	1	Dog	F	18	
1001	1	Cat	M	60	
1002	1	Dog	M	6	
1003	2	Cat	F	36	Delete
1004	2	Dog	U	120	Delete

Result Grid Filter Rows: Search Export:

AnimalID	Species	AgeMonths	Sex	OrgName	ExamDate	HealthScore	Notes
1024	Cat	84	F	South Sound Animal Care	2025-01-27	6	Senior with kidney disease; hydration and labs...
1022	Dog	36	F	South Sound Animal Care	2025-01-25	6	Behavior hold; resource guarding observed; beh...
1020	Dog	96	M	Pawsitive Futures Rescue	2025-01-23	6	Retired working dog; arthritis and hip discomfort.
1011	Dog	84	M	South Sound Animal Care	2025-01-15	6	Senior dog; heart murmur and arthritis; meds pr...
1004	Dog	120	U	Evergreen Pet Haven	2025-01-09	5	Arthritis; started pain management.
1003	Cat	36	F	Evergreen Pet Haven	2025-01-08	6	Chronic cough; scheduled follow up.

Welfare Follow Up Queries

Allows staff to view animals with concerning welfare exam results.

- **API Endpoint:** GET /welfare-followups?species=...
- **SQL Executed:**

```
SELECT
  A.AnimalID,
  A.Species,
  A.AgeMonths,
  A.Sex,
  Org.OrgName,
  W.Date AS ExamDate,
  W.HealthScore,
  W.Notes
FROM WelfareExam W
JOIN Animal A ON W.AnimalID = A.AnimalID
JOIN Organization Org ON W.OrgID = Org.OrgID
WHERE W.HealthScore <= 6
ORDER BY W.Date DESC, A.AnimalID;
```

- **Purpose:**

- Multi-table join
- Identifies animals needing additional medical attention
- Supports species filtering from UI dropdown
- Demonstrates complex JOIN + optional WHERE filtering

In the backend, we optionally add a species filter parameter from the UI dropdown (for example, adding `AND A.Species = ?` when the user selects Dog or Cat).

Advanced Queries (Complex)

These views run multi-table queries from our database to support welfare and management use cases.

Welfare Follow-Ups

Shows animals with welfare exams where the health score is low (≤ 6), joined with their organization.

Species: All Run Welfare Query

Animal ID	Species	Age (months)	Sex	Org	Exam Date	Health Score	Notes
1024	Cat	84	F	South Sound Animal Care	2025-01-27	6	Senior with kidney disease; hydration and labs monitored.
1022	Dog	36	F	South Sound Animal Care	2025-01-25	6	Behavior hold; resource guarding observed; behavior plan.
1020	Dog	96	M	Pawsitive Futures Rescue	2025-01-23	6	Retired working dog; arthritis and hip discomfort.
1011	Dog	84	M	South Sound Animal Care	2025-01-15	6	Senior dog; heart murmur and arthritis; meds prescribed.
1004	Dog	120	U	Evergreen Pet Haven	2025-01-09	5	Arthritis; started pain management.
1003	Cat	36	F	Evergreen Pet Haven	2025-01-08	6	Chronic cough; scheduled follow up.

100%
34:464

Result Grid
Filter Rows:
Search
Export:

AnimalID	Species	AgeMonths	Sex	OrgName	ExamDate	HealthScore	Notes
1024	Cat	84	F	South Sound Animal Care	2025-01-27	6	Senior with kidney disease; hydration and labs...
1022	Dog	36	F	South Sound Animal Care	2025-01-25	6	Behavior hold; resource guarding observed; beh...
1020	Dog	96	M	Pawsitive Futures Rescue	2025-01-23	6	Retired working dog; arthritis and hip discomfort.
1011	Dog	84	M	South Sound Animal Care	2025-01-15	6	Senior dog; heart murmur and arthritis; meds pr...
1004	Dog	120	U	Evergreen Pet Haven	2025-01-09	5	Arthritis; started pain management.
1003	Cat	36	F	Evergreen Pet Haven	2025-01-08	6	Chronic cough; scheduled follow up.

Result 25

Adoption Statistics Queries

Shows adoption patterns grouped by state and species.

- **API Endpoint:** GET /adoption-state?state=...
- **SQL Executed:**

```
SELECT
  L.State,
```

```
A.Species,  
COUNT(*) AS AdoptionCount  
FROM Adoption AD  
JOIN Outcome O ON AD.OutcomeID = O.OutcomeID  
JOIN Animal A ON O.AnimalID = A.AnimalID  
JOIN Adopter D ON AD.AdopterSsn = D.Ssn  
JOIN Location L ON D.LocationID = L.LocationID  
GROUP BY L.State, A.Species  
ORDER BY L.State, A.Species;
```

- **Purpose:**

- Aggregation (COUNT + GROUP BY)
- Multi-table joins
- Manager-view scenario
- Demonstrates complex JOIN + GROUP BY + parameterized filtering

In the backend, we optionally apply a state filter from the UI (for example, adding `WHERE L.State = ?` when the user selects a specific state).

Adoption Stats

Shows adoption counts by state and species using a multi-table join and GROUP BY.

State: All Run Adoption Stats Query

State	Species	Adoption Count
WA	Cat	6
WA	Dog	14

Data source: Pet Health & Welfare MySQL database on Railway.

Result Grid   **Filter Rows:**

	State	Species	AdoptionCou...
	WA	Cat	6
	WA	Dog	14

Result 26

Project Evaluation & Reflection

This project required significant effort across database design, SQL development, backend integration, and UI implementation. Most of the time was spent designing and revising the database schema to ensure that all tables, keys, and constraints worked correctly together. Once the schema was finalized, writing queries and connecting the database to the backend became much easier.

What Went Well:

Once we settled on our final schema and relationships, development progressed smoothly. Using surrogate keys and clearly defining entities such as Intake and Outcome helped simplify joins and queries. Hosting the database on Railway was reliable, and after resolving initial setup issues, the UI, backend, and database communicated consistently. Our SQL queries returned results that matched our expected use cases.

What Went Wrong:

The most challenging part of the project was making schema changes while also building the backend and UI. When the database design changed, we had to update the table definitions, sample data, and queries to keep everything consistent. We also experienced connection issues when attempting to host the database on Azure, which slowed progress and ultimately led us to switch to Railway.

Lessons Learned and Future Improvements:

If we had more time, we would further normalize parts of the schema by separating lookup values into their own tables, such as Species, IntakeType, and OutcomeType, instead of storing them as text fields. This would reduce redundancy and improve consistency. We would also expand CRUD functionality beyond the minimal interface and add additional views to support more detailed reporting and analysis.

References

Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems* (7th ed.). Pearson.

MySQL. (2024). *MySQL 8.0 Reference Manual*. Oracle.

<https://dev.mysql.com/doc/>

Railway. (2024). *Railway Documentation – MySQL Hosting*.

<https://docs.railway.app>

React. (2024). *React Official Documentation*. Meta.

<https://react.dev/learn>

OpenAI. (2024). *ChatGPT Model & API Documentation*.

<https://platform.openai.com/docs/>

Appendices

Appendix A — Full database schema

```
-- 1. ORGANIZATION
CREATE TABLE Organization (
  OrgID      INT          NOT NULL,
  OrgName    VARCHAR(100) NOT NULL,
  Type       VARCHAR(100) NOT NULL,
  CONSTRAINT PK_Organization PRIMARY KEY (OrgID),
  CONSTRAINT UQ_Organization_OrgName UNIQUE (OrgName),
  CONSTRAINT CK_Organization_OrgID CHECK (OrgID >= 1)
);

-- 2. LOCATION
CREATE TABLE Location (
  LocationID INT          NOT NULL,
  City       VARCHAR(50)  NOT NULL,
  State      CHAR(2)      NOT NULL,
  Description VARCHAR(500) NULL,
  OrgID      INT          NOT NULL,
  CONSTRAINT PK_Location PRIMARY KEY (LocationID),
  CONSTRAINT FK_Location_Org
    FOREIGN KEY (OrgID) REFERENCES Organization (OrgID),
  CONSTRAINT CK_Location_LocationID CHECK (LocationID >= 1)
);

-- 3. ANIMAL
CREATE TABLE Animal (
  AnimalID   INT          NOT NULL,
  OrgID      INT          NOT NULL,
  Species    VARCHAR(30)  NOT NULL,
  Sex        CHAR(1)      NOT NULL,
  AgeMonths  INT          NOT NULL,
  Microchip  VARCHAR(20)  NULL,
  Notes      VARCHAR(500) NULL,
  CONSTRAINT PK_Animal PRIMARY KEY (AnimalID),
  CONSTRAINT FK_Animal_Org
    FOREIGN KEY (OrgID) REFERENCES Organization (OrgID),
  CONSTRAINT UQ_Animal_Microchip UNIQUE (Microchip),
  CONSTRAINT CK_Animal_AnimalID CHECK (AnimalID >= 1),
  CONSTRAINT CK_Animal_Sex CHECK (Sex IN ('M', 'F', 'U')),
  CONSTRAINT CK_Animal_AgeMonths CHECK (AgeMonths BETWEEN 0 AND 1000)
);

-- 4. INTAKE (weak entity under Animal; composite PK)
CREATE TABLE Intake (
  IntakeNo    INT          NOT NULL,
  AnimalID    INT          NOT NULL,
  IntakeType   VARCHAR(100) NOT NULL,
  IntakeDateTime TIMESTAMP NOT NULL,
  SourceText   VARCHAR(500) NULL,
  LocationID  INT          NOT NULL,
```

```

CONSTRAINT PK_Intake PRIMARY KEY (IntakeNo, AnimalID),
CONSTRAINT FK_Intake_Animal
    FOREIGN KEY (AnimalID) REFERENCES Animal (AnimalID),
CONSTRAINT FK_Intake_Location
    FOREIGN KEY (LocationID) REFERENCES Location (LocationID),
CONSTRAINT CK_Intake_IntakeNo CHECK (IntakeNo >= 1)
);

-- 5. OUTCOME
CREATE TABLE Outcome (
    OutcomeID INT NOT NULL,
    AnimalID INT NOT NULL,
    OutcomeType VARCHAR(100) NOT NULL,
    OutcomeDateTime TIMESTAMP NOT NULL,
    Returned BOOLEAN NOT NULL DEFAULT FALSE,
    IsLive BOOLEAN NOT NULL,
    CONSTRAINT PK_Outcome PRIMARY KEY (OutcomeID),
    CONSTRAINT FK_Outcome_Animal
        FOREIGN KEY (AnimalID) REFERENCES Animal (AnimalID),
    CONSTRAINT CK_Outcome_OutcomeID CHECK (OutcomeID >= 1)
);

-- 6. ADOPTER
CREATE TABLE Adopter (
    Ssn CHAR(9) NOT NULL,
    Phone VARCHAR(20) NOT NULL,
    Email VARCHAR(150) NOT NULL,
    FirstName VARCHAR(100) NOT NULL,
    LastName VARCHAR(100) NOT NULL,
    LocationID INT NOT NULL,
    CONSTRAINT PK_Adopter PRIMARY KEY (Ssn),
    CONSTRAINT FK_Adopter_Location
        FOREIGN KEY (LocationID) REFERENCES Location (LocationID)
);

-- 7. ADOPTION (specialization of Outcome)
CREATE TABLE Adoption (
    OutcomeID INT NOT NULL,
    AdopterSsn CHAR(9) NOT NULL,
    AdoptionDate DATE NOT NULL,
    AdoptionFee DECIMAL(8,2) NOT NULL DEFAULT 0.00,
    CONSTRAINT PK_Adoption PRIMARY KEY (OutcomeID),
    CONSTRAINT FK_Adoption_Outcome
        FOREIGN KEY (OutcomeID) REFERENCES Outcome (OutcomeID),
    CONSTRAINT FK_Adoption_Adopter
        FOREIGN KEY (AdopterSsn) REFERENCES Adopter (Ssn)
);

-- 8. WELFARE EXAM
CREATE TABLE WelfareExam (
    WelfareExamID INT NOT NULL,
    Date DATE NOT NULL,
    Weight DECIMAL(5,2) NULL,
    HealthScore INT NULL,
    Notes VARCHAR(500) NULL,

```

```

OrgID          INT          NOT NULL,
AnimalID       INT          NOT NULL,
CONSTRAINT PK_WelfareExam PRIMARY KEY (WelfareExamID),
CONSTRAINT FK_WelfareExam_Org
    FOREIGN KEY (OrgID) REFERENCES Organization (OrgID),
CONSTRAINT FK_WelfareExam_Animal
    FOREIGN KEY (AnimalID) REFERENCES Animal (AnimalID),
CONSTRAINT CK_WelfareExam_WelfareExamID CHECK (WelfareExamID >= 1),
CONSTRAINT CK_WelfareExam_HealthScore
    CHECK (HealthScore IS NULL OR (HealthScore BETWEEN 1 AND 10))
);

```

Appendix B — All INSERT statements

```

-----
-- 1. ORGANIZATION
-----
INSERT INTO Organization (OrgID, OrgName, Type)
VALUES
(1, 'Seattle Animal Rescue',      'Municipal Shelter'),
(2, 'Evergreen Pet Haven',        'Nonprofit Rescue'),
(3, 'Pawsitive Futures Rescue',    'Nonprofit Rescue'),
(4, 'South Sound Animal Care',     'Municipal Shelter');

-----
-- 2. LOCATION
-----
INSERT INTO Location (LocationID, City, State, Description, OrgID)
VALUES
(100, 'Seattle',    'WA', 'Main Shelter - Downtown',      1),
(101, 'Seattle',    'WA', 'Foster Home - Northgate',       1),
(102, 'Everett',    'WA', 'Everett City Park (found in bush)', 1),
(103, 'Bellevue',   'WA', 'Adopter Home - Single Family',   2),
(104, 'Tacoma',     'WA', 'Adopter Apartment Building',     2),

(105, 'Tacoma',     'WA', 'Main Shelter - Tacoma',          4),
(106, 'Kent',       'WA', 'Satellite Clinic - Kent',         1),
(107, 'Redmond',    'WA', 'Foster Home - Redmond',          3),
(108, 'Everett',    'WA', 'Shelter Annex - Everett',        3),
(109, 'Olympia',    'WA', 'Adopter Home - Rural Property',   4),
(110, 'Seattle',    'WA', 'Neighborhood foster home - Ballard', 1),
(111, 'Renton',     'WA', 'Renton satellite shelter',       1),
(112, 'Everett',    'WA', 'Evergreen Pet Haven main shelter', 2),
(113, 'Lynnwood',   'WA', 'Partner vet clinic',             2),
(114, 'Kirkland',   'WA', 'Foster apartment',              3),
(115, 'Puyallup',   'WA', 'South Sound intake facility',     4),
(116, 'Tacoma',     'WA', 'Offsite adoption event location', 4),
(117, 'Shoreline',  'WA', 'Short-term foster home',         1),
(118, 'Bellevue',   'WA', 'High-volume adopter neighborhood', 2);

-----
-- 3. ANIMAL
-----
INSERT INTO Animal (AnimalID, OrgID, Species, Sex, AgeMonths, Microchip, Notes)

```

```

VALUES
(1000, 1, 'Dog', 'F', 18, 'MC1000A', 'Friendly, came in as stray. '),
(1001, 1, 'Cat', 'M', 60, 'MC1001B', 'Owner surrender; shy in kennel. '),
(1002, 1, 'Dog', 'M', 6, NULL, 'Found in park with no collar. '),
(1003, 2, 'Cat', 'F', 36, 'MC1003D', 'Transferred from another rescue. '),
(1004, 2, 'Dog', 'U', 120, 'MC1004E', 'Senior dog with mobility issues. '),

(1005, 1, 'Dog', 'F', 24, 'MC1005F',
'High-energy dog; needs training and large yard. '),
(1006, 1, 'Cat', 'F', 8, NULL,
'Kitten with upper respiratory infection; on meds. '),
(1007, 2, 'Dog', 'M', 36, 'MC1007G',
'Good with other dogs; loves fetch. '),
(1008, 3, 'Cat', 'M', 48, 'MC1008H',
'Bonded pair with AnimalID 1009. '),
(1009, 3, 'Cat', 'F', 48, 'MC1009I',
'Bonded pair with AnimalID 1008. '),
(1010, 4, 'Dog', 'F', 4, NULL,
'Puppy surrendered from accidental litter. '),
(1011, 4, 'Dog', 'M', 84, 'MC1011J',
'Large-breed senior; heart murmur noted. '),
(1012, 2, 'Cat', 'U', 18, NULL,
'Semi-feral barn cat; prefers outdoor placement. '),

(1013, 1, 'Dog', 'M', 30, 'MC1013K',
'Medium mixed breed; dog-friendly and crate trained. '),
(1014, 1, 'Cat', 'F', 72, 'MC1014L',
'Declawed front paws; must be indoor only. '),
(1015, 1, 'Dog', 'U', 10, NULL,
'Shy puppy; still learning leash; scared of loud noises. '),
(1016, 2, 'Cat', 'M', 24, 'MC1016M',
'Young adult; black coat; tolerant of handling. '),
(1017, 2, 'Dog', 'F', 60, 'MC1017N',
'Bonded with AnimalID 1018; surrendered together. '),
(1018, 2, 'Dog', 'M', 60, 'MC1018O',
'Bonded with 1017; nervous around men. '),
(1019, 3, 'Cat', 'F', 15, NULL,
'Polydactyl kitten; very social; good candidate for family home. '),
(1020, 3, 'Dog', 'M', 96, 'MC1020P',
'Retired working dog; arthritis in hips; needs low-impact exercise. '),
(1021, 3, 'Cat', 'U', 5, NULL,
'Stray kitten; underweight; on bottle feeding initially. '),
(1022, 4, 'Dog', 'F', 36, 'MC1022Q',
'Behavior hold; resource guarding around food. '),
(1023, 4, 'Dog', 'M', 20, NULL,
'Recovered from parvo; currently on medical hold. '),
(1024, 4, 'Cat', 'F', 84, 'MC1024R',
'Senior cat; chronic kidney disease; special diet. '),
(1025, 2, 'Cat', 'M', 40, NULL,
'Barn cat candidate; prefers outdoor or semi-feral placement. '),
(1026, 1, 'Dog', 'F', 14, 'MC1026S',
'Small breed; brought in with puppies; very people focused. '),
(1027, 1, 'Dog', 'M', 8, NULL,
'Puppy from 1026's litter; playful and mouthy. '),
(1028, 1, 'Dog', 'F', 8, NULL,

```

```

    'Puppy from same litter as 1027; slightly more reserved.'),
(1029, 3, 'Cat', 'M', 30, 'MC1029T',
    'Returned adoption; inappropriate urination reported in home.'),
(1030, 4, 'Dog', 'M', 48, 'MC1030U',
    'Large hound; strong puller on leash; scent-driven.');
```

```
-- 4. ADOPTER
-----
```

```
INSERT INTO Adopter (Ssn, Phone, Email, FirstName, LastName, LocationID)
VALUES
('123456789', '206-555-0101', 'alice.nguyen@example.com', 'Alice', 'Nguyen', 103),
('234567890', '425-555-0202', 'brian.lee@example.com', 'Brian', 'Lee', 104),
('345678901', '360-555-0303', 'carla.santos@example.com', 'Carla', 'Santos', 103),

('456789012', '425-555-0404', 'david.kim@example.com', 'David', 'Kim', 103),
('567890123', '509-555-0505', 'emily.chan@example.com', 'Emily', 'Chan', 104),
('678901234', '253-555-0606', 'frank.garcia@example.com', 'Frank', 'Garcia', 109),
('789012345', '206-555-0707', 'grace.wong@example.com', 'Grace', 'Wong', 103),
('890123456', '425-555-0808', 'hannah.ross@example.com', 'Hannah', 'Ross', 114),
('901234567', '360-555-0909', 'ian.miller@example.com', 'Ian', 'Miller', 118),
('012345678', '253-555-1010', 'julia.park@example.com', 'Julia', 'Park', 111),
('098765432', '206-555-1111', 'kevin.tran@example.com', 'Kevin', 'Tran', 117);

-----
```

```
-- 5. WELFARE EXAM
-----
```

```
INSERT INTO WelfareExam (WelfareExamID, Date, Weight, HealthScore, Notes, OrgID,
AnimalID)
VALUES
(3000, '2025-01-05', 45.20, 8, 'Initial intake exam; mild skin irritation.', 1,
1000),
(3001, '2025-01-06', 9.10, 7, 'Dental tartar noted.', 1,
1001),
(3002, '2025-01-07', 12.30, 9, 'Puppy in good condition.', 1,
1002),
(3003, '2025-01-08', 7.80, 6, 'Chronic cough; scheduled follow up.', 2,
1003),
(3004, '2025-01-09', 55.00, 5, 'Arthritis; started pain management.', 2,
1004),

(3005, '2025-01-10', 52.00, 7,
    'Slight anxiety in kennel; overall healthy.', 1, 1005),
(3006, '2025-01-11', 2.10, 8,
    'Kitten underweight but improving on treatment.', 1, 1006),
(3007, '2025-01-12', 40.50, 9,
    'Fit and active; cleared for adoption events.', 2, 1007),
(3008, '2025-01-13', 5.20, 8,
    'Bonded pair exam for 1008; mild dental tartar.', 3, 1008),
(3009, '2025-01-13', 5.00, 8,
    'Bonded pair exam for 1009; body condition normal.', 3, 1009),
(3010, '2025-01-14', 6.80, 9,
    'Puppy exam; vaccines started, no issues.', 4, 1010),
(3011, '2025-01-15', 75.30, 6,
    'Senior dog; heart murmur and arthritis; meds prescribed.', 4, 1011),
```

```

(3012, '2025-01-16', 4.40, 7,
'Semi-feral; stressy in kennel; good body weight.', 2, 1012),
(3013, '2025-01-17', 28.40, 8,
'Healthy; mild tartar; cleared for adoption events.', 1, 1013),
(3014, '2025-01-18', 4.50, 7,
'Senior cat; weight slightly low; kidney values monitored.', 1, 1014),
(3015, '2025-01-19', 9.20, 8,
'Puppy exam; boosters started; slightly nervous.', 1, 1015),
(3016, '2025-01-20', 5.10, 9,
'Young cat; no major issues; vaccinated and microchipped.', 2, 1016),
(3017, '2025-01-21', 55.00, 7,
'Bonded pair exam with 1018; moderate dental disease.', 2, 1017),
(3018, '2025-01-21', 54.50, 7,
'Bonded pair exam with 1017; nervous but handleable.', 2, 1018),
(3019, '2025-01-22', 3.20, 8,
'Kitten; dewormed; extra toes noted.', 3, 1019),
(3020, '2025-01-23', 68.00, 6,
'Retired working dog; arthritis and hip discomfort.', 3, 1020),
(3021, '2025-01-24', 1.80, 7,
'Bottle baby; gaining weight slowly; needs frequent feeds.', 3, 1021),
(3022, '2025-01-25', 40.20, 6,
'Behavior hold; resource guarding observed; behavior plan.', 4, 1022),
(3023, '2025-01-26', 22.50, 7,
'Post-parvo recovery; GI signs resolved; recheck in 1 week.', 4, 1023),
(3024, '2025-01-27', 4.10, 6,
'Senior with kidney disease; hydration and labs monitored.', 4, 1024),
(3025, '2025-01-28', 4.30, 8,
'Barn cat candidate; healthy; minimal handling.', 2, 1025),
(3026, '2025-01-29', 6.20, 9,
'Postpartum dog; body condition good; puppies nursing.', 1, 1026),
(3027, '2025-01-30', 2.50, 9,
'Healthy puppy; growing well; dewormed.', 1, 1027),
(3028, '2025-01-30', 2.40, 9,
'Littermate of 1027; healthy and social.', 1, 1028),
(3029, '2025-02-01', 4.90, 7,
'Returned adoption; mild cystitis suspected.', 3, 1029),
(3030, '2025-02-02', 70.00, 8,
'Large hound; strong but otherwise healthy.', 4, 1030);

-----
-- 6. INTAKE (PK = IntakeNo + AnimalID)
-----

INSERT INTO Intake (IntakeNo, AnimalID, IntakeType, IntakeDateTime, SourceText,
LocationID)
VALUES
(1, 1000, 'Stray', '2025-01-04 15:32:00',
'Found roaming near downtown shelter; brought in by AC officer.', 100),
(1, 1001, 'Owner Surrender', '2025-01-04 10:05:00',
'Owner moving and unable to keep cat.', 100),
(1, 1002, 'Stray', '2025-01-05 13:20:00',
'Good Samaritan found dog near playground.', 102),
(1, 1003, 'Transfer In', '2025-01-06 09:45:00',
'Transferred from partner rescue in Spokane.', 100),
(1, 1004, 'Public Assist', '2025-01-06 16:10:00',
'Owner requested help rehoming senior dog.', 101),

```

```

(2, 1000, 'Return from Adoption', '2025-02-10 11:15:00',
  'Adopter returned due to housing restrictions.', 100),

(1, 1005, 'Stray', '2025-01-09 12:05:00',
  'Found running loose near dog park; no ID tags.', 106),
(1, 1006, 'Stray', '2025-01-10 09:40:00',
  'Found under parked car; sneezing and congested.', 100),
(1, 1007, 'Owner Surrender', '2025-01-10 15:25:00',
  'Owner could not afford ongoing vet costs.', 104),
(1, 1008, 'Seizure', '2025-01-11 10:10:00',
  'Removed from neglect case; housed together with 1009.', 108),
(1, 1009, 'Seizure', '2025-01-11 10:10:00',
  'Removed from same neglect case as 1008; bonded pair.', 108),
(1, 1010, 'Owner Surrender', '2025-01-12 14:22:00',
  'Accidental litter; owner surrendered 3 puppies, kept 1.', 105),
(1, 1011, 'Public Assist', '2025-01-13 17:05:00',
  'Neighbor reported elderly dog tied outside for long periods.', 105),
(1, 1012, 'Stray', '2025-01-14 08:30:00',
  'Trapped as part of barn cat relocation program.', 102),

(1, 1013, 'Stray', '2025-01-16 13:00:00',
  'Found loose near neighborhood park; no ID.', 110),
(1, 1014, 'Owner Surrender', '2025-01-17 10:15:00',
  'Owner moving to assisted living; cannot keep cat.', 100),
(1, 1015, 'Stray', '2025-01-18 18:25:00',
  'Puppy found under porch; no microchip found.', 111),
(1, 1016, 'Stray', '2025-01-19 09:40:00',
  'Found near apartment complex; friendly with people.', 112),
(1, 1017, 'Owner Surrender', '2025-01-20 14:05:00',
  'Bonded pair surrendered due to divorce.', 112),
(1, 1018, 'Owner Surrender', '2025-01-20 14:05:00',
  'Same bonded pair surrender; came in together with 1017.', 112),
(1, 1019, 'Stray', '2025-01-21 11:55:00',
  'Kitten brought in by Good Samaritan.', 114),
(1, 1020, 'Transfer In', '2025-01-22 15:30:00',
  'Transfer from working dog program.', 108),
(1, 1021, 'Stray', '2025-01-23 08:10:00',
  'Found in alley; very underweight.', 114),
(1, 1022, 'Seizure', '2025-01-24 16:45:00',
  'Animal control seized dog from neglect case.', 115),
(1, 1023, 'Owner Surrender', '2025-01-25 10:00:00',
  'Owner unable to afford treatment after parvo.', 115),
(1, 1024, 'Owner Surrender', '2025-01-26 13:20:00',
  'Owner moving and cannot continue medical care.', 113),
(1, 1025, 'Stray', '2025-01-27 07:50:00',
  'Trapped on farm; part of barn cat program.', 113),
(1, 1026, 'Owner Surrender', '2025-01-28 12:05:00',
  'Owner surrendered mom dog and litter.', 111),
(1, 1027, 'Litter Intake', '2025-01-28 12:05:00',
  'Puppy from same litter as 1026; processed individually.', 111),
(1, 1028, 'Litter Intake', '2025-01-28 12:05:00',
  'Second puppy from same litter; processed individually.', 111),
(1, 1029, 'Adoption Return', '2025-01-31 10:40:00',
  'Returned due to litter box issues.', 110),
(1, 1030, 'Stray', '2025-02-01 17:30:00',

```

```

        'Found following jogger; extremely energetic.',          116),

(2, 1002, 'Return from Adoption', '2025-03-01 11:45:00',
        'Returned due to mismatched energy level with adopter.', 100),
(2, 1022, 'Return from Foster',   '2025-02-15 09:30:00',
        'Behavior concerns reported; returned from foster.',    115),
(2, 1029, 'Stray',                '2025-03-05 13:15:00',
        'Seen again roaming neighborhood; readmitted.',        110);

-----
-- 7. OUTCOME
-----

INSERT INTO Outcome (OutcomeID, AnimalID, OutcomeType, OutcomeDateTime, Returned,
IsLive)
VALUES
(2000, 1000, 'Adopted',          '2025-01-20 14:00:00', FALSE, TRUE),
(2001, 1001, 'Transferred',      '2025-01-18 09:30:00', FALSE, TRUE),
(2002, 1002, 'Adopted',          '2025-01-25 16:45:00', FALSE, TRUE),
(2003, 1003, 'Adopted',          '2025-02-01 11:00:00', FALSE, TRUE),
(2004, 1004, 'Euthanized',       '2025-02-05 17:20:00', FALSE, FALSE),
(2005, 1000, 'Returned to Shelter', '2025-02-10 11:00:00', TRUE, TRUE),

(2006, 1005, 'Adopted',          '2025-02-01 13:10:00', FALSE, TRUE),
(2007, 1006, 'Foster',           '2025-01-20 16:30:00', FALSE, TRUE),
(2008, 1007, 'Adopted',          '2025-02-10 10:05:00', FALSE, TRUE),
(2009, 1008, 'Adopted',          '2025-02-15 14:45:00', FALSE, TRUE),
(2010, 1009, 'Adopted',          '2025-02-15 14:45:00', FALSE, TRUE),
(2011, 1010, 'Adopted',          '2025-02-20 09:20:00', FALSE, TRUE),
(2012, 1011, 'Transferred',      '2025-02-25 17:55:00', FALSE, TRUE),
(2013, 1012, 'Relocated',        '2025-03-01 08:10:00', FALSE, TRUE),
(2014, 1002, 'Adopted',          '2025-03-10 15:40:00', FALSE, TRUE),

(2015, 1013, 'Adopted',          '2025-02-05 14:10:00', FALSE, TRUE),
(2016, 1014, 'Transferred',      '2025-02-07 09:00:00', FALSE, TRUE),
(2017, 1015, 'Foster',           '2025-02-08 16:20:00', FALSE, TRUE),
(2018, 1016, 'Adopted',          '2025-02-10 11:05:00', FALSE, TRUE),
(2019, 1017, 'Adopted',          '2025-02-12 13:45:00', FALSE, TRUE),
(2020, 1018, 'Adopted',          '2025-02-12 13:45:00', FALSE, TRUE),
(2021, 1019, 'Adopted',          '2025-02-14 10:25:00', FALSE, TRUE),
(2022, 1020, 'Transferred',      '2025-02-16 15:30:00', FALSE, TRUE),
(2023, 1021, 'Foster',           '2025-02-18 09:50:00', FALSE, TRUE),
(2024, 1022, 'Behavior Hold',    '2025-02-20 17:05:00', FALSE, TRUE),
(2025, 1023, 'Adopted',          '2025-02-22 14:40:00', FALSE, TRUE),
(2026, 1024, 'Hospice Foster',   '2025-02-24 11:15:00', FALSE, TRUE),
(2027, 1025, 'Relocated',        '2025-02-26 08:30:00', FALSE, TRUE),
(2028, 1026, 'Adopted',          '2025-02-28 10:10:00', FALSE, TRUE),
(2029, 1027, 'Adopted',          '2025-03-01 12:05:00', FALSE, TRUE),
(2030, 1028, 'Adopted',          '2025-03-01 12:05:00', FALSE, TRUE),
(2031, 1029, 'Adopted',          '2025-03-10 16:40:00', TRUE, TRUE),
(2032, 1030, 'Adopted',          '2025-03-12 15:20:00', FALSE, TRUE);

-----
-- 8. ADOPTION (only for OutcomeType = 'Adopted')
-----

INSERT INTO Adoption (OutcomeID, AdopterSsn, AdoptionDate, AdoptionFee)

```



```
VALUES
(2000, '123456789', '2025-01-20', 150.00),
(2002, '234567890', '2025-01-25', 200.00),
(2003, '345678901', '2025-02-01', 75.00),

(2006, '456789012', '2025-02-01', 175.00), -- 1005
(2008, '567890123', '2025-02-10', 225.00), -- 1007
(2009, '678901234', '2025-02-15', 120.00), -- 1008
(2010, '678901234', '2025-02-15', 120.00), -- 1009 bonded pair
(2011, '789012345', '2025-02-20', 250.00), -- 1010
(2014, '123456789', '2025-03-10', 200.00), -- 1002 second adoption

(2015, '890123456', '2025-02-05', 180.00), -- 1013
(2018, '901234567', '2025-02-10', 95.00), -- 1016
(2019, '456789012', '2025-02-12', 250.00), -- 1017
(2020, '456789012', '2025-02-12', 250.00), -- 1018
(2021, '098765432', '2025-02-14', 130.00), -- 1019
(2025, '012345678', '2025-02-22', 220.00), -- 1023
(2028, '123456789', '2025-02-28', 175.00), -- 1026
(2029, '234567890', '2025-03-01', 200.00), -- 1027
(2030, '234567890', '2025-03-01', 200.00), -- 1028
(2031, '678901234', '2025-03-10', 75.00), -- 1029
(2032, '789012345', '2025-03-12', 210.00); -- 1030
```

Appendix C — All bad data tests

```
-- 1) Duplicate Microchip (UNIQUE constraint on Animal.Microchip)
INSERT INTO Animal (AnimalID, OrgID, Species, Sex, AgeMonths, Microchip)
VALUES (999, 1, 'Dog', 'M', 10, 'MC1000A');
-- Expected: FAIL (duplicate microchip value)

-- 2) Invalid Sex value (domain constraint)
INSERT INTO Animal (AnimalID, OrgID, Species, Sex, AgeMonths, Microchip)
VALUES (1000, 1, 'Cat', 'X', 8, NULL);
-- Expected: FAIL (Sex must be 'M', 'F', or 'U')

-- 3) Intake referencing non-existent AnimalID (FK violation)
INSERT INTO Intake (IntakeNo, AnimalID, IntakeType, IntakeDateTime, LocationID)
VALUES (99, 9999, 'Stray', '2025-01-01 10:00:00', 100);
-- Expected: FAIL (AnimalID 9999 does not exist)

-- 4) Outcome referencing non-existent AnimalID (FK violation)
INSERT INTO Outcome (OutcomeID, AnimalID, OutcomeType, OutcomeDateTime, Returned,
IsLive)
VALUES (999, 9999, 'Adopted', '2025-02-01 12:00:00', FALSE, TRUE);
-- Expected: FAIL (AnimalID 9999 does not exist)

-- 5) Adoption referencing non-existent OutcomeID (FK violation)
INSERT INTO Adoption (OutcomeID, AdopterSsn, AdoptionDate, AdoptionFee)
VALUES (9999, '123456789', '2025-03-01', 50.00);
-- Expected: FAIL (OutcomeID 9999 does not exist)

-- 6) Delete an Outcome that is referenced by an Adoption (should fail)
DELETE FROM Outcome WHERE OutcomeID = 2000;
```

```
-- Expected: FAIL (OutcomeID 2000 is referenced by Adoption)
```

Appendix D — All SQL queries used in UI + demo

A. Simple Read Query (Animals List)

Used by the Animals page to display a table of animals.

```
SELECT AnimalID, OrgID, Species, Sex, AgeMonths, Microchip, Notes
FROM Animal
ORDER BY AnimalID
LIMIT 50;
```

B. Create (Insert Animal)

Example query used when the “Add Animal” form is submitted.

```
INSERT INTO Animal (AnimalID, OrgID, Species, Sex, AgeMonths, Microchip, Notes)
VALUES (101, 1, 'Dog', 'F', 24, '985112345678901', 'Friendly, good with kids');
```

(In the actual UI, these values are filled in from the form inputs.)

C. Delete (Remove Animal)

Example query used when deleting an animal by ID.

```
DELETE FROM Animal
WHERE AnimalID = 101;
```

(In the UI, the specific AnimalID comes from the selected row.)

D. Species Aggregation (UI Summary Chips)

Used to show how many animals of each species are currently in the system.

```
SELECT Species, COUNT(*) AS Count
FROM Animal
GROUP BY Species;
```

E. Complex Query #1 — Welfare Follow-Ups

Returns animals with low health scores and their most recent welfare exams.

```
SELECT
    A.AnimalID,
    A.Species,
    A.AgeMonths,
    A.Sex,
    Org.OrgName,
    W.Date AS ExamDate,
    W.HealthScore,
    W.Notes
FROM WelfareExam W
JOIN Animal A      ON W.AnimalID = A.AnimalID
JOIN Organization Org ON W.OrgID = Org.OrgID
WHERE W.HealthScore <= 6
ORDER BY W.Date DESC, A.AnimalID;
```

In the UI, we optionally filter by species (e.g., only Dogs or Cats).

That filter is handled in the backend code, so here we show the core MySQL query.

F. Complex Query #2 — Adoption Stats by State and Species

Summarizes how many adoptions occurred by state and animal species.

```
SELECT
  L.State,
  A.Species,
  COUNT(*) AS AdoptionCount
FROM Adoption AD
JOIN Outcome O   ON AD.OutcomeID = O.OutcomeID
JOIN Animal A    ON O.AnimalID = A.AnimalID
JOIN Adopter D   ON AD.AdopterSSN = D.SSN
JOIN Location L  ON D.LocationID = L.LocationID
GROUP BY L.State, A.Species
ORDER BY L.State, A.Species;
```

In the live system, this query can be filtered by a specific state (e.g., only 'WA'); the filter is implemented in the backend, but the base aggregation is shown here.